Detecting LLM-Associated Expressions Through Longitudinal GitHub Language Change

`load-bearing` — technical summary

August 2026

Abstract. An unsupervised pipeline that finds expressions whose use on GitHub rises unusually fast, groups those that rise *together*, and uses the resulting lexicon to score text and to estimate when a passage of model-written prose was generated. The premise is that a model generation leaves a *bundle* of habits rather than one catchphrase, so the unit of evidence is a group of expressions moving at the same time. From 4.92M GitHub documents over 103 months the system recovers 62,828 candidate expressions and 45 temporal clusters, dates LLM-written text to a mean absolute error of 5.6 months against a 9.3-month baseline, and measures declared AI assistance rising from zero to 50 commits per 10,000. Two headline results are negative and reported as such: cluster timing cannot be distinguished from a randomly shifted release calendar, and date estimation does not transfer reliably to a generator the model has not seen.

---

1 Data collection

Source. GH Archive, the hourly public event log of GitHub. Unauthenticated, complete back to 2015, and the only longitudinal record of GitHub prose that does not require replaying the API.

Sampling. The archive is roughly 70 MB per hour, so a census is impractical. Hours are sampled on a deterministic diagonal: days spread across the month, hours spread across the clock, so each month's sample covers both the weekly and the diurnal cycle, and the pattern is identical every month so that months stay comparable. Sampling density rises with era (4 hours/month for the 2018–2021 baseline, 5 for 2022–2025, 40 for 2026, where the archive is degraded).

Streaming. Raw archive files are never written to disk. Each worker downloads one hour into memory, applies a byte-level prefilter to the head of each line — `type` is the second JSON key, so the 60–90% of events that are pushes are rejected without being parsed — cleans the surviving prose, and writes a small Parquet shard. 34 GB streamed through RAM produced 564 MB on disk.

Yield. 686 hour-slots, one unavailable; 4,921,372 documents across issues, issue comments, pull-request descriptions, PR comments, review comments and review bodies. Median 38,841 documents per month.

1.1 Two archive defects that shape every result

The archive degrades partway through the window, and both defects imitate the signal being hunted:

- **Pull-request descriptions vanish from 2025-11.** The artifact mix shifts hard (`issue_comment` falls from 35% to 12% of the corpus over the window; `pr_review_comment` rises from 13% to 28%). Untreated, every expression commoner in review comments than in descriptions shows a jump exactly where the mix moved — 21 of 30 clusters initially placed their shared changepoint in that month.

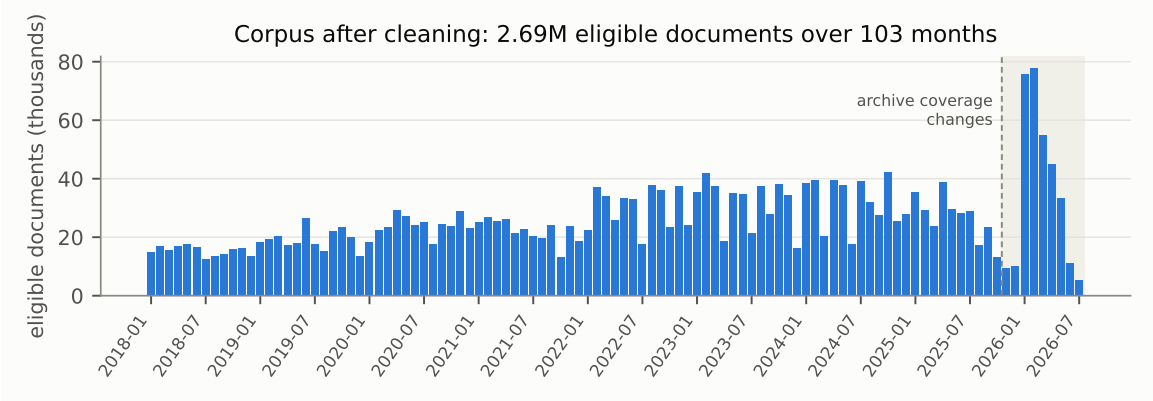

Figure 1. Eligible documents per month after cleaning. The shaded window is where the archive stops carrying pull-request descriptions; comparisons that straddle its boundary measure the collector rather than the language.

- **Commit payloads vanish from 2025-10.** `PushEvent` records carry no `commits` array at all: 0 of 20,113 sampled in 2025-12, against 10,464 of 10,473 in 2025-09. Commit-message analysis therefore stops at 2025-09.

2 Pre-processing

Everything here exists because a specific artefact was found masquerading as emergent language.

Cleaning. Fenced and indented code, logs and stack traces, URLs, quoted replies, HTML, markdown tables, mentions, issue references and SHAs are removed. Runs of two or more machine-output lines are dropped, while isolated matches are kept — prose legitimately mentions `foo.py:12`.

One eligibility predicate. The same function decides both numerator and denominator of every rate: ≥ 8 tokens, English by a function-word test, less than 90% of characters removed as code. Any asymmetry here makes frequencies move with the document mix rather than with language. 2.69M of 4.92M documents qualify.

Machine authors are excluded, and counted. The machine-authored share of GitHub prose runs from 7% (2018) to 37% (2025) to 53% (2026). Left in, the study would measure how many review bots comment on GitHub. The bot flag is recomputed from the author login at analysis time, never read from the shard: the list of AI review tools grows faster than any snapshot of it, and GitHub's own reviewer posts as plain `copilot` with no `[bot]` suffix.

Boilerplate is mined, not listed. Whole-document de-duplication misses the damaging case: a banner inserted into text a human really wrote. Lines recurring verbatim across ≥ 5 repositories in a period are mined and stripped, with digit runs masked so *reviewed 67 of 69 files* and *reviewed 3 of 4 files* collapse to one template. Lines under five tokens are exempt, so `lgtm` survives as language. A complementary detector flags authors whose documents share a modal six-token opening across repositories — this catches tools that are templated in shape but unique in content, which nothing else finds.

Near-duplicates. Banded LSH over 64-permutation MinHash signatures of 5-gram shingles; clusters confined to one repository are local templates, clusters spanning several are platform-wide.

Documents are truncated to 400 tokens for feature extraction. Document frequency gives every document one vote, but the median document is 30 tokens while generated project write-ups run past 1000 and carry thousands of distinct n-grams. A few dozen of those push many expressions over threshold at once, and those expressions then co-emerge perfectly — because they are literally the same documents.

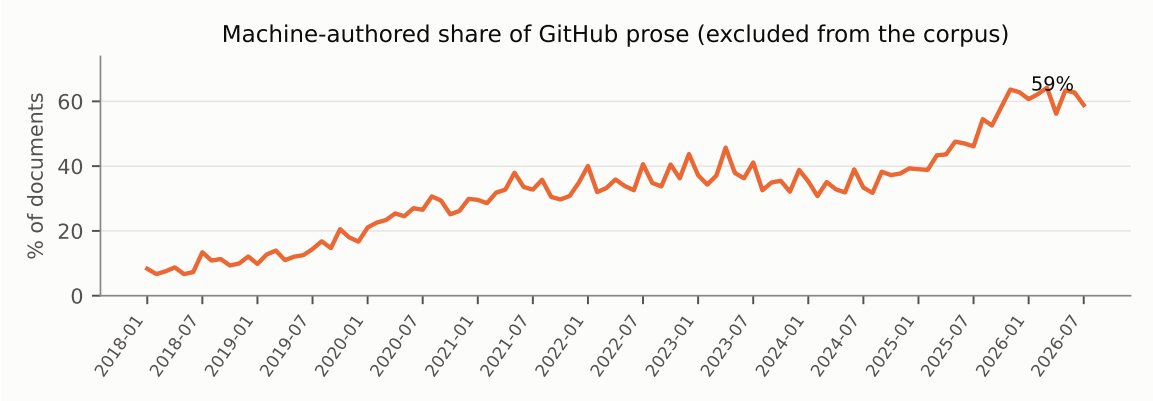

Figure 2. Share of documents written by identifiable machine authors. These are excluded from the discovery corpus and reused as the training population for date estimation (§5).

Rates are standardised over a fixed artifact mix. Direct standardisation: the rate within each artifact type is estimated separately, then recombined with weights fixed at the corpus average, so a period's own mix cannot move any expression's rate. Periods whose artifact coverage falls below 90% of the corpus average are dropped rather than compared.

3 Model

3.1 Features

Counting is by *document frequency*: an expression is present or absent, never counted twice, so one author repeating a phrase eight times is one adoption. Four feature kinds share a namespace:

- Word 1–3-grams over a canonical token stream where hyphens and apostrophes are dissolved, so `load-bearing` and `load bearing` are the same bigram. n-grams never cross clause boundaries.
- Hyphenated compounds (`HYPH:load-bearing`) and typography markers (`TYPO:em_dash`), which carry their own signal.
- About 50 hand-written rhetorical constructions (`CONSTR:not_just_but`), for frames whose slots vary and which no n-gram captures.
- Character n-grams, available but off by default.

Inflectional and surface variants collapse into families via a conservative singulariser (WordNet where installed, an explicit irregular table otherwise) applied only for grouping, never for counting: inflection is itself stylistic signal.

3.2 Three passes

Width and cost trade off, so the wide passes stay cheap. **Pass A** counts every feature in every period and keeps what clears a document-frequency floor. **Vocabulary selection** pools into calendar windows — a per-period floor makes visibility depend on that period's volume, biasing discovery against thin months — and admits expressions with large variation in *either* direction. **Pass B** recounts the 62,828 candidates exactly in all 103 periods, stratified by artifact type. **Pass C** adds repositories, authors, contexts, capitalisation and cohesion for a 4,000-expression shortlist.

3.3 Emergence

For each expression the single best level shift is found by maximising a binomial z over every admissible split, fully vectorised across expressions. With k documents containing the expression

and n eligible documents,

$$z(s) = \frac{k_{\text{post}} - n_{\text{post}}\,\hat{p}_{\text{pre}}}{\sqrt{n_{\text{post}}\,\hat{p}_{\text{pre}}(1-\hat{p}_{\text{pre}})}}, \qquad \hat{p}_{\text{pre}} = \max\left(\frac{k_{\text{pre}}}{n_{\text{pre}}}, \frac{0.5}{n_{\text{pre}}}\right).$$

This is the right test for rare events, where a rate can triple on a handful of documents. A second, robust z against a rolling median with MAD scale asks whether the jump is unusual *for that expression*. An expression only the first statistic likes is rare and lucky; one only the second likes is noisy. Both are reported.

The emergence score is a bounded sum of saturating terms — growth, significance, robustness, persistence, rise speed, prior rarity — rather than a product, so no single component can carry an expression to the top. It is gated on the shift being upward: the best split is a maximum even for an expression that only ever fell. Declines stay in the table, because a rise-then-fall profile is what pins down a narrow date.

3.4 Clusters as the unit of evidence

Expressions are compared by growth series $g_{e,t} = \Delta \log(f_{e,t} + \varepsilon)$ under a similarity combining correlation, changepoint agreement and small-lag cross-correlation. Average-linkage clustering is followed by a rank-one fit $g_{e,t} \approx a_e L_{k,t}$, giving each cluster a latent curve and each member a loading. Clusters must span ≥ 3 distinct *families*: three inflections of one phrase are one habit, not three witnesses.

Confounders are removed before clustering, not after. Bot templates and migration imports co-emerge perfectly by construction; filtering afterwards leaves them dominating the output, and the largest cluster in an early run was 209 members of market-research spam. Five measures do the work: repository concentration, repository spread, AI-repository share, artifact-mix entropy (graded, not thresholded — spam filed as issue bodies is broad across repositories and defeats every other test), mid-sentence capitalisation (a phrase almost always capitalised is a product name, and *any* capitalised word in the phrase counts), and document cohesion, the mean pairwise Jaccard among the documents containing the expression — high cohesion means the expression marks a *kind of document* rather than a way of writing.

4 Inference

Text scoring. Expression weights combine emergence strength, confounder penalty, breadth and cluster loading. Two guards prevent one striking phrase carrying a verdict: only the strongest member of a family counts, and within-cluster evidence is pooled concavely (contribution $= (\sum w)^{0.6}$) while independent clusters add linearly. A document with three expressions from one 2025 cluster therefore scores well above one phrase but well below three unrelated markers. The output is a continuous LLM-era expression score, never a human-versus-model verdict: humans who read model output adopt its phrasing too.

Date estimation. $P(t \mid d) \propto P(t) \prod_e P(e \mid t)$, with two departures from the textbook form, both forced by measurement:

- *Present expressions contribute a rate ratio, not a raw probability.* Absence looks informative, but over a fixed feature set it is swamped by document length — a two-sentence comment lacks almost every expression whenever it was written. Using absence dated recent documents years early, with error growing the more recent the document. The ratio form is invariant to how many features are absent.
- *Each period's rate is shrunk toward the expression's global rate* with fixed strength in document-equivalents. Add-half smoothing puts a floor of $0.5/n_t$ on every cell, an order of magnitude

higher in a thin month than a fat one, which drags estimates toward whichever months were sampled most heavily.

A cluster-level variant collapses each cluster to one observation, since members that rose together carry nearly the same information and treating twelve as twelve observations produces absurdly sharp posteriors.

Likelihoods come from the population being dated. $P(e \mid t)$ estimated on general GitHub prose describes how developers write; dating model output needs how *generated* text read in that month, and a phrase can be ubiquitous in model output while rare in human comments. The estimator is therefore fitted on the material the rest of the pipeline discards: 298,318 documents from 16 review bots and coding agents, which are dated and unambiguously LLM-written.

Feature selection for dating is a separate problem from discovery. Emergence ranks by size of jump from a low base, so its top expressions are rare by construction — only 29 of them appeared in even 0.4% of LLM documents, leaving two-thirds of test documents with no evidence. Dating features are instead selected from the full recounted vocabulary by mutual information between presence and period, measured on LLM text, with a support floor scaled to corpus size.

5 Benchmark

Every test is written so that it can fail, and so that failure is legible.

- **Temporal backtest.** Fit before a cutoff; ask whether the rise *holds* afterwards, not whether it keeps rising — an expression that entered the language plateaus, so "still rising" penalises the cases the detector should get right. Windows either side of the cutoff are equal length, or the comparison measures which years were sampled most.
- **Repository holdout.** Discover on one hash-partition of repositories, verify on a disjoint one, so a phrase one large project uses cannot replicate by accident.
- **Placebo releases.** Null A shifts the release calendar *circularly* inside the observed window; null B scatters the changepoints. A plain shift pushes the calendar out of the data range and loses by construction, which made an early version report $p = 0.000$ for a result that does not hold. The test also reports its own power.
- **Pre-era placebo.** Run the whole detector on pre-2022 data alone. Language changed before LLMs, and that is the noise floor any claim must clear.
- **Cluster stability.** Re-cluster under perturbed thresholds, aggregation and period sets; measure how often pairs stay together.
- **Date recovery.** Hide timestamps and measure error in months, stratified by period so the most heavily sampled months cannot dominate, with a repository split and a leave-one-generator-out condition.

6 Results

6.1 Discovery

62,828 candidate expressions; 4,000 measured for breadth; 426 survive the confounder filter; 45 clusters of ≥ 3 expressions spanning ≥ 3 families. Cluster changepoints concentrate in 2025-02 to 2025-06 (14 clusters). Representative members:

Cluster	Emerged	Members
0	2025-06	`coding agent`, `copilot instructions`, `by claude`, `coding agents`, `fallback chain`, `cause identified`
3	2025-02	`an mcp server`, `in the mcp`, `tool-call`, `handoffs`, `with clear error`
6	2025-04	`for claude code`, `human-in-the`, `correctly implements`, `returns structured`
2, 5	2018-09/10	`a clear and concise description`, `add any other context about the`

The 2018 clusters are GitHub's issue-template rollout — a real event, correctly dated, and evidence that the detector is not simply finding LLM-shaped things everywhere.

6.2 Most representative expressions, last three months

Comparing 2026-05–07 (49,790 documents) against 2025-11–2026-04 (273,013), inside one artifact-coverage regime. Ranked by evidence contributed per document (Bernoulli KL: prevalence × distinctiveness).

Expression	% of docs	growth	repos
em dash (—)	29.1	3.0×	8,613
`so the`	6.7	4.0×	2,808
`gate`	4.1	6.3×	1,621
arrow (→)	9.5	2.6×	3,755
`surface`	3.9	5.3×	1,664
`scope / out of scope`	7.5	3.0×	2,922
`verified`	5.0	3.5×	2,040
`follow-up`	4.0	3.8×	1,603
`canonical`	2.1	4.8×	903
`load-bearing`	0.4	13.9×	194
`byte-identical`	0.3	20.8×	167
`git diff --check`	0.3	36.9×	97

The em dash is the strongest single marker by a factor of 2.8 in information contributed, appearing in 29% of recent documents against 10% before, across 8,613 repositories. **But the top five expressions carry only 12% of the total evidence and the top ten 19%**: the signal is genuinely diffuse, which supports the bundle premise and means no single expression is diagnostic. The contexts place these in one register — structured agentic verification reports pasted into pull-request discussions — which explains why `load-bearing` keeps company with `byte-identical` and `git diff --check`.

6.3 Declared AI assistance

Coding agents append a trailer to what they help write, which is a declared label on text submitted under a *human* account — the only ground truth available for that middle category. Detection must run before cleaning, since the cleaner strips the trailer deliberately, and the marker is kept only as a column: left in the document, every expression in an assisted commit would correlate perfectly with its own label.

Across 252,655 commits, Claude Code trailers per 10,000 commits: zero through 2025-03, then 1.8 (2025-04), 20.2 (2025-06), 39.8 (2025-08), 50.4 (2025-09). First appearance coincides with the tool's release rather than preceding it. This is a *floor*, not an estimate: only assistance that the tool declares and the author did not strip is visible.

A first version of the generic pattern matched any co-author line containing *ai* or *bot*, making that category 18× larger than Claude Code's while being almost entirely Renovate, Dependabot, pre-commit-ci and github-actions. Reporting it would have claimed ∼1% AI-assisted commits

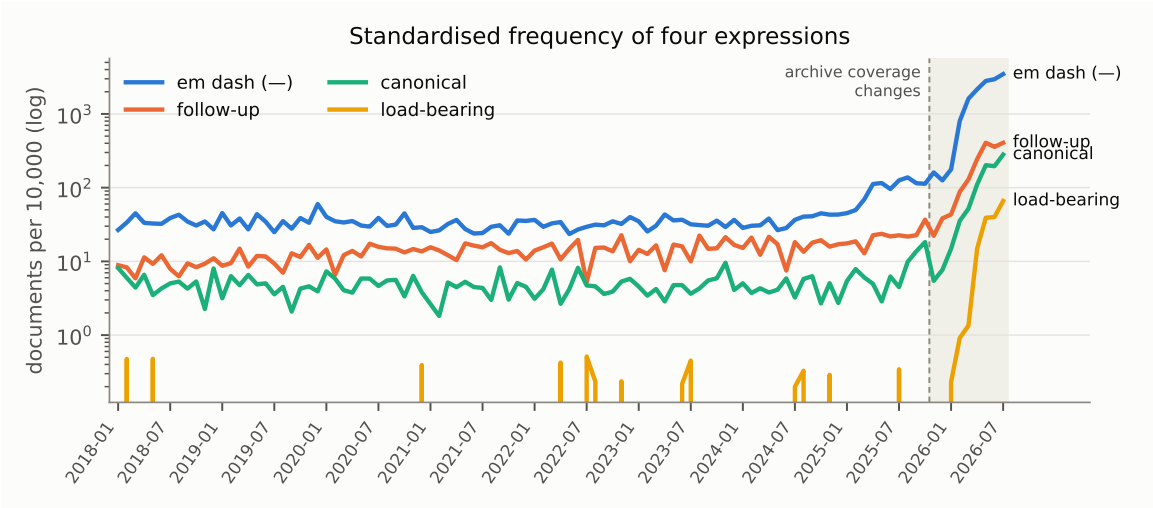

Figure 3. Four expressions on a standardised log scale. All four turn upward from mid-2025; the steepest segment falls inside the changed-coverage window, which is why the recent-window analysis is run inside a single regime rather than against history.

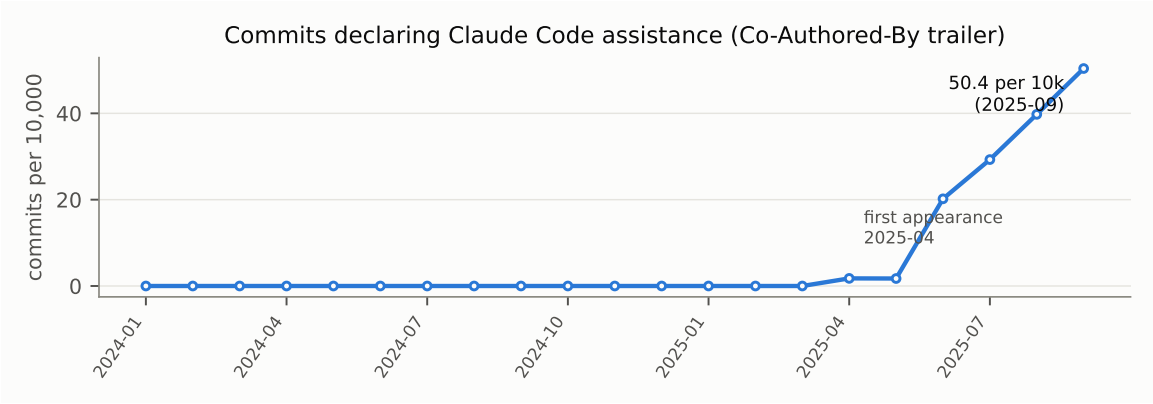

Figure 4. Commits declaring Claude Code assistance. Zero for fifteen months, then a 28× rise over six. The series ends at 2025-09 because the archive stops carrying commit payloads.

that were really dependency automation.

6.4 Dating LLM-written text

Fitted on 194,236 LLM-authored training documents over 38 months, tested on a disjoint repository partition:

Condition	MAE	Baseline	Skill	Coverage
All generators, repository holdout	5.56 mo	9.30	1.67×	100%
Cluster-level likelihood	5.55 mo	9.30	1.68×	100%
Held out: Copilot	6.20	8.98	1.45×	100%
Held out: CodeRabbit	10.59	8.73	0.82×	100%
Held out: Gemini Code Assist	12.33	9.73	0.79×	100%

Median error 3 months; 53% of documents within 3 months, 72% within 6. **Transfer to an unseen generator fails for two of three tools**, so most of the in-distribution skill is generator-specific style rather than a universal "when was this generated" signal. The method will date text from a generator it has seen; treat it as unproven on one it has not.

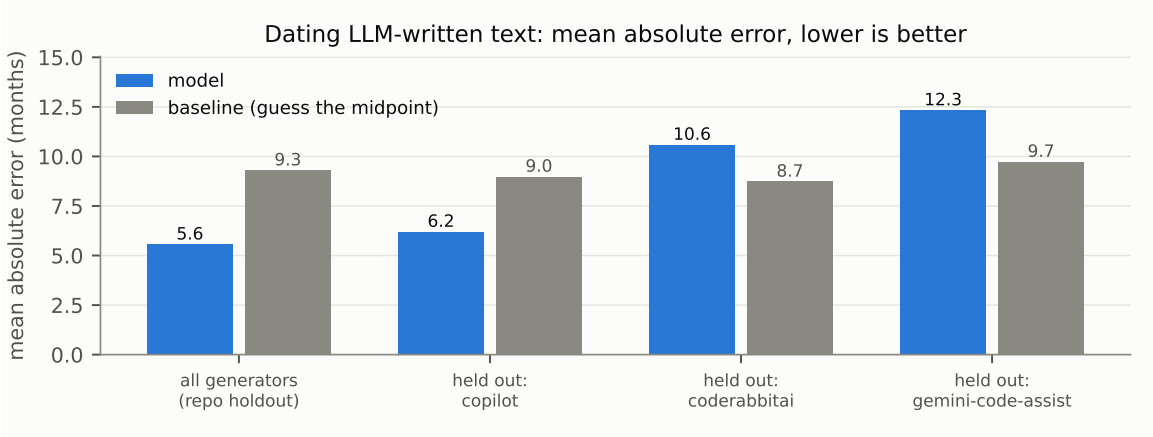

Figure 5. Date-recovery error. The model beats the midpoint baseline overall and on held-out Copilot, and loses on held-out CodeRabbit and Gemini Code Assist.

6.5 Validation

Test	Result
Pre-era placebo	**8.0**× enrichment of LLM-era emergences above the pre-era 99th percentile. Clears the noise floor by a wide margin.
Cluster stability	88.8% of co-clustered pairs hold in a majority of six settings.
Repository holdout	97% of rises replicate in both partitions, but log-ratio correlation is only 0.23 — weak agreement on *magnitude*.
Placebo releases	**Inconclusive.** $p = 0.69$ (shifted calendar), $p = 0.80$ (shuffled changepoints); 19 releases in a 93-month window leave a 4.9-month mean gap, so any date lands near some release.
Temporal backtest	**Negative.** Flagged rises retain 1.03× their level out of sample against 1.19× for volume-matched controls. Partly winner's curse — selection on a jump implies regression to the mean — but not isolated.

7 Limitations

- **Release attribution does not work.** Cluster timing is consistent with the release calendar but indistinguishable from a shifted one. At this release density the test has no power; separating specific models was never in scope, and even generation-level attribution is unsupported here.
- **Cross-generator date transfer is unproven**, and the leave-one-out result suggests generator identity does much of the work.
- **The archive limits the recent window.** PR descriptions end 2025-11 and commit payloads 2025-10; the last three months are interpretable only against neighbours inside the same regime.
- **Declared assistance is a floor.** Undeclared assistance is invisible, and the declaring population is self-selected.
- **The subject is drifting.** Much of the recent signal is agent-generated or agent-pasted text under human accounts. That is arguably what "LLM-era language" means, but it is not the claim that humans changed how they write unprompted.
- **A high score is not attribution.** It says a text uses expressions that spread in a period, nothing about who or what produced it.

Implementation: 14 modules, 41 tests. `lbdetect ingest` → `lbdetect all` reproduces every number above; `lbdetect recent`, `lbdetect provenance` and `lbdetect date-model` produce §6.2, §6.3 and §6.4.